



Resumen operativo: migracion de Gurobi a PuLP

Alcance

Esta adaptacion parte del repositorio:

```
references/repos/refuting_a_widespread_belief_about_county_splits
```

La version modificada vive en:

```
references/adaptations/refuting-county-splits
```

El codigo original esta bajo GPL-3.0. La adaptacion conserva la licencia y la atribucion en `LICENSE` y `NOTICE`. El repositorio original se mantiene sin modificaciones.

La meta fue conservar las propuestas algoritmicas del articulo usando PuLP, sin intentar simular las APIs privadas de Gurobi.

Estructura final

```
refuting-county-splits/  
|- README.md  
|- LICENSE  
|- NOTICE  
|- requirements.txt  
|- requirements-optional.txt  
|- refuting_county_splits/  
|   |- __init__.py  
|   |- data.py  
|   |- metrics.py  
|   |- optimization.py  
|   `-- serialization.py  
|- scripts/  
|   |- reproduce.py  
|   `-- run_tabasco.py  
|- examples/  
|   `-- synthetic.json  
`- tests/  
    `-- test_synthetic.py
```

El entorno aislado de esta referencia se encuentra en:

```
references/adaptations/refuting-county-splits/.venv
```

Dependencias principales: `networkx` , `pulp` y, opcionalmente, `geopandas` .

Modelo original y traduccion

El articulo trabaja sobre un grafo no dirigido $G=(V, E)$:

- `V` : unidades geograficas.
- `E` : pares de unidades adyacentes.
- `p_i` : poblacion de cada unidad.
- `k` : numero total de distritos.
- `k'` : numero de distritos representados por el cluster buscado.

- L, U : limites de poblacion por distrito.

El modelo original usa dos variables binarias por nodo:

```
x[i,0] = 1 si i pertenece al cluster S  
x[i,1] = 1 si i pertenece al complemento V-S
```

La adaptacion usa una sola variable:

```
x[i] = 1 si i pertenece al cluster S
```

El complemento se representa como $1-x[i]$.

Algoritmo replicado: clusters top-t

Ubicacion

```
refuting_county_splits/optimization.py::enumerate_top_districts
```

Formulacion matematica

Para un cluster de tamaño k' :

```
L*k' <= sum(p_i*x_i) <= U*k'  
L*(k-k') <= sum(p_i*(1-x_i)) <= U*(k-k')
```

La raíz r debe pertenecer al cluster:

```
x_r = 1
```

Contiguidad por flujo

Se crea flujo dirigido sobre cada arista. Cada nodo seleccionado, excepto la raíz, consume una unidad:

```
sum_j f[j,i] - sum_j f[i,j] = x_i      para i != r
f[i,j] <= M*x_i
```

con $M=|V|-1$. Esto reproduce el esquema de flujo de Shirabe del repositorio original.

Aristas cortadas

Para cada arista $\{i,j\}$ se define $\text{cut}[i,j]$:

```
x_i - x_j <= cut[i,j]
x_j - x_i <= cut[i,j]
```

Con variables binarias, $\text{cut}[i,j]=1$ cuando la arista cruza la frontera del cluster.

Para cut_edges se minimiza:

```
min sum(cut[i,j])
```

Para perimeter , si existen datos geometricos, se minimiza:

```
min sum(shared_perim[i,j]*cut[i,j])
    + sum(boundary_perim[i]*x_i)
```

Enumeracion en PuLP

Gurobi encontraba soluciones sucesivas dentro de callbacks. PuLP no expone callbacks MIP equivalentes, por lo que la adaptacion ejecuta un ciclo externo:

1. Construye y resuelve el MILP.
2. Recupera el cluster seleccionado.
3. Comprueba con NetworkX que el complemento sea conexo.
4. Si no es conexo, agrega una desigualdad separadora y vuelve a resolver.
5. Si es valido, guarda el cluster.
6. Agrega un no-good cut para excluir esa asignacion.

7. Repite hasta obtener t soluciones o llegar a infactibilidad.

`find_minimal_separator` implementa la búsqueda de la frontera separadora usada para construir los cortes del complemento.

El artículo expresa el separador con variables del complemento. Como la adaptación almacena solo x_i , utiliza:

```
(1-x_a) + (1-x_b)
<= 1 + sum(1-x_c for c in separator)
```

Algoritmo replicado: heurística de tallado

Ubicación

`refuting_county_splits/optimization.py::districting_heuristic`

Idea matemática

Si P es un plan parcial, se define:

```
V' = V - union(D in P) D
k' = k - |P|
```

La heurística busca hasta t distritos candidatos en V' , agrega cada uno al plan parcial y repite. Cuando quedan $k-1$ distritos, asigna todos los nodos restantes al último distrito.

El número máximo teórico de ramas es aproximadamente $t^{(k-1)}$.

La heurística no garantiza:

- enumeración exhaustiva;
- mínimo global de `cut_edges`;
- mínimo global de divisiones;
- igualdad con el conjunto de planes de Gurobi;

- optimalidad global del plan completo.

Es una búsqueda dirigida por objetivos locales.

Metricas replicadas

Se encuentran en `refuting_county_splits/metrics.py` :

- `cut_edges` .
- `perimeter` .
- `inverse_polsby_popper` .
- `district_metrics` .
- `plan_objective` .

Para un distrito con perimetro `P` y area `A` :

```
inverse_PP = P^2 / (4*pi*A)
```

El upstream modelaba Polsby-Popper con la restriccion cuadratica:

```
P^2 <= 4*pi*A*objective
```

PuLP trabaja aqui con MILP lineal. Por eso, `inverse_polsby_popper` es una metrica posterior y el modelo usa el proxy lineal de perimetro.

Datos y carga

`refuting_county_splits/data.py` contiene:

- `load_adjacency_graph` : carga JSON NetworkX del upstream.
- `normalize_population` : acepta `TOTPOP` , `P0010001` , `population` o `total_population` .
- `load_municipios_csv` : convierte los CSV de Tabasco a `networkx.Graph` .

El solver solo necesita `TOTPOP` , nodos y aristas; no depende de nombres Census.

Que no se pudo replicar literalmente

Callbacks y separacion dinamica

No se pudieron trasladar literalmente:

- MIPSOL .
- MIPNODE .
- MIPNODE_OBJBND .
- cbLazy .
- terminacion temprana basada en bound.

Se sustituyeron por restricciones agregadas entre resoluciones.

MIQCP de Polsby-Popper

La restriccion cuadratica no fue trasladada a PuLP. Polsby-Popper quedo como metrica posterior y el perimetro como proxy de optimizacion.

Planes finales de Alabama y North Carolina

Se replica la generacion de clusters, pero no un pipeline automatico de los planes finales. En el upstream, la seleccion compatible de clusters y la division final en distritos fueron parcialmente manuales.

Exhaustividad

No se implemento `enumpart` ni la enumeracion exhaustiva de todos los planes. El tallado solo explora las ramas top-t seleccionadas.

Verificacion

Se ejecutaron:

```
ruff check .  
PYTHONPATH=. .venv/bin/pytest -q
```

Resultado: 6 passed .

La adaptacion es una replica operacional y educativa, no una equivalencia de rendimiento con Gurobi.